Computers & Security 148 (2025) 104132

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Edge-featured multi-hop attention graph neural network for intrusion detection system

Ping Deng, Yong Huang *

College of Computer Science, Sichuan Normal University, Chengdu, China

ARTICLE INFO

Keywords:
Multi-hop attention
Graph neural networks
Intrusion detection
Internet of Things

ABSTRACT

With the development of the Internet, the application of computer technology has rapidly become widespread, driving the progress of Internet of Things (IoT) technology. The attacks present on networks have become more complex and stealthy. However, traditional network intrusion detection systems with singular functions are no longer sufficient to meet current demands. While some machine learning-based network intrusion detection systems have emerged, traditional machine learning methods cannot effectively respond to the complex and dynamic nature of network attacks. Intrusion detection systems utilizing deep learning can better enhance detection capabilities through diverse data learning and training. To capture the topological relationships in network data, using graph neural networks (GNNs) is most suitable. Most existing GNNs for intrusion detection use multi-layer network training, which may lead to over-smoothing issues. Additionally, current intrusion detection solutions often lack efficiency. To mitigate the issues mentioned above, this paper proposes an Edge-featured Multi-hop Attention Graph Neural Network for Intrusion Detection System (EMA-IDS), aiming to improve detection performance by capturing more features from data flows. Our method enhances computational efficiency through attention propagation and integrates node and edge features, fully leveraging data characteristics. We carried out experiments on four public datasets, which are NF-CSE-CIC-IDS2018-v2, NF-UNSW-NB15-v2, NF-BoT-IoT, and NF-ToN-IoT. Compared with existing models, our method demonstrated superior performance.

1. Introduction

As the Internet of Things (IoT) advances, numerous IoT devices have been integrated into the Internet platform. Although IoT devices make our lives increasingly convenient, the risk of cyber attacks on these devices also increases (Peng et al., 2021). Cyber attacks have become more complex, with Advanced Persistent Threats (APTs) and botnets being predominant. Intrusions targeting devices such as firewalls, routers, printers, and cameras have become critical in breaching defense lines (He et al., 2023). Currently, network intrusion detection systems (NIDS) (Heidari and Jabraeil Jamali, 2023) can be implemented using two main schemes: traditional rule-based signature detection and machine learning-based methods. Rule-based detection systems require pre-set filtering rules, which are inadequate for today's rapidly evolving network environment. In contrast, machine learning-based intrusion detection systems can learn the characteristics of various attacks, making them more intelligent and efficient.

Many well-known machine learning methods have been applied in network intrusion detection systems, such as SVM, random forest, decision tree, and XGBoost (Thakkar and Lohiya, 2022). These traditional machine learning algorithms initially played an excellent role in intrusion detection but heavily relied on feature extraction engineering. With the successful application of deep learning methods in the fields of natural language processing and image recognition (Sajjad et al., 2022; Treviso et al., 2023; Xu et al., 2023; Bayoudh, 2023), network intrusion detection systems have also started to incorporate classic deep learning methods, such as CNN, LSTM, and RNN, to enhance recognition accuracy and efficiency.

Traditional deep learning algorithms use feature statistics and protocol fields to construct classifiers for network traffic (Yi et al., 2023), but such methods do not fully utilize the topological relationships in network space. In recent years, Graph Neural Networks (GNNs) have been widely used (Shao et al., 2024), as they have significantly contributed to breakthroughs in the pharmaceutical and medical technology industries. GNNs are particularly prevalent in applications such as molecular linkage prediction (Torres et al., 2023) and knowledge graph mining (Zhong et al., 2023), where they capture objects' spatial topology. Similarly, data flows can be viewed as directed graphs composed of IP

* Corresponding author.
E-mail addresses: dpenging@stu.sicnu.edu.cn (P. Deng), huangyong@sicnu.edu.cn (Y. Huang).

https://doi.org/10.1016/j.cose.2024.104132
Received 11 June 2024; Received in revised form 29 August 2024; Accepted 20 September 2024
Available online 26 September 2024

addresses and ports in network security. Therefore, employing Graph Neural Networks in network intrusion detection systems can be highly beneficial.

In Network Intrusion Detection Systems (NIDS), the main focus is processing various data streams (such as NetFlow). NetFlow data streams contain source IP, port, destination IP, port, protocol fields, and packet data (Sarhan et al., 2022). By deploying NIDS with machine learning models, it becomes possible to detect the data flowing through in real-time, eliminating the hassle of manually configuring specific filtering rules as done previously. Graph Neural Networks (GNNs) have begun to be applied in network intrusion detection. One of the most well-known classic models is the E-GraphSAGE model proposed by Lo et al. (2022), which uses GraphSAGE (Hamilton et al., 2017) for training. This model aggregates edge information to predict the categories of adjacent node edges. However, GraphSAGE is merely an extension of Graph Convolutional Networks (GCN) (Wu et al., 2019). Although it alleviates some drawbacks of GCN in handling large-scale graphs, it still does not focus on the influence of neighboring nodes on the current node when aggregating node features. Therefore, Graph Attention Networks (GAT) have emerged.GAT (Velickovic et al., 2017) utilizes an attention mechanism, allowing each node to dynamically learn the importance of neighboring nodes and calculate attention weights based on the similarity between neighboring nodes. In the context of Network Intrusion Detection Systems (NIDS), some attempts have been made to use GAT to address issues. For instance, Nguyen and Kashef (2023) proposed a self-supervised model using GAT, significantly improving the detection accuracy. However, typical graph neural networks integrate and update node and edge features in the graph structure through domain aggregation and message passing. Additionally, single-layer graph neural networks only focus on one-hop neighboring nodes. Compared to convolutional networks, graph neural networks can only stack a few layers; otherwise, issues like over-smoothing and over-squashing may occur (Qureshi et al., 2023). Therefore, the common practice is to set up GNNs with two layers. While some studies stack GNNs multiple times (Li et al., 2021), excessively increasing the model layers are inefficient in practice, and the resulting accuracy improvement is minimal.

To address these issues and improve the detection accuracy and efficiency of Network Intrusion Detection Systems (NIDS), we propose a novel Edge-featured Multi-hop Attention Graph Neural Network. By utilizing multi-hop attention, our model enables nodes to focus on the features of neighboring nodes and obtain global information. Our approach simplifies the model structure and considers local and global node information, capturing the entire network topology. In summary, this paper makes the following three contributions:

- We propose a novel graph neural network model called EMA-IDS for network intrusion detection. This method uses local and global attention to effectively aggregate node features and combine them with edge features, resulting in a smaller model size while enhancing detection accuracy and operational efficiency.
- Unlike existing anomaly detection algorithms, our proposed model with multi-hop attention effectively utilizes both node and edge features. As far as we know, this is the first instance where multi-hop attention has been applied to network intrusion detection.
- We conducted extensive experiments and compared our model with existing models on four publicly available datasets. The results show that our approach surpasses existing Graph Neural Network models.

The rest of the paper is structured as follows: Section 2 surveys related work in network intrusion detection. Section 3 provides the essential background information. Section 4 explains the proposed model in detail. Section 5 outlines the experimental setup and presents the results. Section 6 concludes the paper and discusses future work.

2. Related work

2.1. ML-based NIDSs

In recent years, machine learning techniques have been extensively applied in Network Intrusion Detection Systems (NIDS). Numerous researchers have concentrated on integrating machine learning models into NIDS. Churcher et al. (2021) studied the detection performance of various machine learning algorithms in NIDS, including naive Bayes (NB), k-nearest neighbors (KNN), decision trees (DT), support vector machines (SVM) and random forests (RF). Their experimental results indicated that KNN achieved the best performance. Sarhan et al. (2021) provided several NetFlow-based datasets to assess the generalization ability of machine learning models. These datasets are now widely recognized as benchmarks for evaluating the effectiveness of AI techniques in NIDS (Network Intrusion Detection Systems). Lawal et al. (2020) employed the XGBoost algorithm to propose an anomaly mitigation framework for anomaly detection, demonstrating excellent detection performance. Balyan et al. (2022) employed Random Forest combined with EGA-PSO methods to address the overfitting problem caused by data imbalance, thereby enhancing feature extraction and detection accuracy. In Gu and Lu (2021) and Al-Yaseen (2019), the SVM algorithm was used for different tasks to detect anomalous traffic, which also enhanced the detection robustness of Intrusion Detection Systems (IDS). Jin et al. (2020) proposed a SwiftIDS model, which efficiently processes network traffic data and performs well on large-scale datasets. When introduced as AI methods into intrusion detection systems, these previous classical machine learning methods opened a new door for the field. They demonstrated that intrusion detection can benefit from machine learning support, becoming more intelligent in addition to signature-based rule methods.

2.2. DL-based NIDSs

Deep learning is also widely applied as a branch of machine learning. Given the complexity of feature engineering, deep learning is used in various tasks. Deep learning-based IDS are now prevalent, as deep learning can learn more features through multiple hidden layers. These methods are trained using both supervised and unsupervised approaches. Convolutional Neural Networks (CNN), initially utilized for image recognition, have now been extensively studied for anomaly detection (ElSayed et al., 2021; Kanna and Santhi, 2021; Sun et al., 2020; Zhang et al., 2020). These methods extract spatial features from data using CNNs or integrating CNNs with Long Short-Term Memory (LSTM) networks. Researchers have used LSTM models (Greff et al., 2016; Laghrissi et al., 2021; Imrana et al., 2021) to effectively leverage spatial features for classifying network data, achieving higher accuracy, fewer errors, and better classification metrics on public datasets (UNSW-NB15, ISCX-IDS, and NSL-KDD). Recurrent Neural Networks (RNNs) (Shi et al., 2017) are extensively used in NIDS. They are specifically designed to process sequential data, including text and time series. Their internal structure includes recurrent connections, enabling the model to capture temporal dependencies within the sequences. In Sohi et al. (2021), Sohi et al. were the first to demonstrate that using RNNs helps generate new, unseen attack variants and synthetic signatures from state-of-the-art malware, thus improving intrusion detection rates. Haghighat and Li (2021) introduced a voting-based deep learning framework incorporating a voting engine to select the most accurate prediction candidates during the prediction phase, effectively reducing false positives in anomaly detection. Devendiran and Turukmane (2024) recently developed a Gated Attention Dual Long Short-Term Memory Network (Dugat-LSTM) for attack classification, showcasing excellent accuracy and robustness. In Rani et al. (2024), Talukder et al. (2024) and Turukmane and Devendiran (2024), various deep learning models were presented, employing normalization or principal component analysis (PCA) to standardize data. The experimental results consistently demonstrated that these approaches enhanced detection accuracy.

2.3. GNN-based NIDSs

Relevant research has already emerged in 2022. Lo et al. (2022) were the first to apply Graph Neural Networks (GNNs) to intrusion detection systems. Their approach involved using the GraphSAGE model to construct graphs from datasets and predict the edge types between adjacent nodes using a Multi-Layer Perceptron (MLP). This method is both efficient and innovative. However, it has some drawbacks: randomly generating nodes before graph construction can improve generalization performance but at the cost of losing the authenticity of public datasets. Additionally, their method's effectiveness could weaken when dealing with large-scale data. Caville et al. (2022) modified the E-GraphSAGE model and proposed the DGI model. Although it introduced a self-supervised module, the overall process of the model is relatively cumbersome and lacks simplicity. Therefore, in 2023, Nguyen and Kashef (2023) proposed a traffic-aware self-supervised model. This model uses a linear classifier to determine whether a node is anomalous, with the criterion for anomaly being that a node is considered anomalous if its traffic is above the average. While leveraging traffic awareness is a good idea, this method also has a high false positive rate. Xu et al. (2024) introduced a new NEGSC model, which also employed a self-supervised approach. In terms of detection performance, this model outperformed previous baseline models. Duan et al. (2024) proposed a Continuous Temporal Graph Convolutional (CTG) detection framework to address the challenge of detecting new connected entities in fixed topologies, particularly in the context of 5G/B5G networks. Rehman et al. (2024) employed a Word2Vec-based semantic encoder to capture key semantic attributes in provenance graphs and designed a lightweight classifier that combines Graph Neural Networks (GNN) and Word2Vec (Grohe, 2020) embeddings. Their approach also demonstrated robustness and scalability. This represents another avenue of exploration.

2.4. Over-smoothing issue on GNN

The use of Graph Neural Networks (GNNs) has become increasingly popular in the field of network intrusion detection. However, the over-smoothing phenomenon has also impacted the accuracy of detection tasks. As the number of layers increases, the performance of the model becomes unstable, primarily because deep GNN models tend to lose local information (Qureshi et al., 2023; Chen et al., 2020; Xhonneux et al., 2020). Many recent studies have aimed at addressing this issue. Xu et al. (2018) proposed JK-Net, which improves performance by adding connections between layers, aggregating information from each layer to the final layer and ensuring that low-dimensional information is retained, leading to remarkable performance in related experiments. Sun et al. (2019) introduced another model, Adagcn, which iteratively integrates feature embeddings from each layer using the Adaboost algorithm, bringing low-order features into the deep layers and accumulating node predictions from all layers. In Chen et al. (2023), a new graph structural layer called GEL was proposed, which uses residual connections and embedding aggregation structures to aggregate high-quality node representations from neighboring nodes through an improved Adaboost algorithm. Manocchio et al. (2024) implemented a Transformer-based network intrusion detection system, though it did not fully utilize the network topology information as it was entirely based on Transformers. Currently, Graph Transformers are gaining traction. Unlike conventional GATs, Graph Transformers combine the advantages of Transformers and graph structures, enabling them to handle complex relational structures in graph data. Huang et al. (2024) proposed Subtree Attention effectively mitigated the over-smoothing issue, improving performance while reducing time complexity. This situation motivates us to explore the application of Graph Transformers in network intrusion detection.

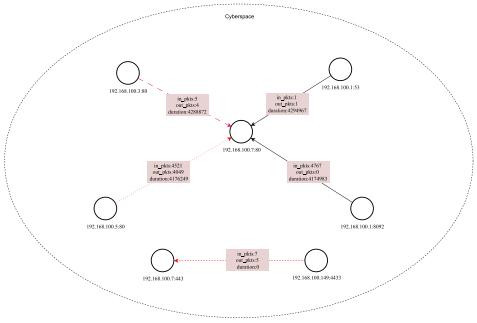

Fig. 1. Depicts nodes in cyberspace identified by IP and port numbers, with edges representing requests sent from the source node to the target node. The edge data includes in_pkts, out_pkts, duration, and other attributes. Black edges indicate normal requests, while red dashed edges represent various attacks.

3. Background

In this section, we present some fundamental concepts utilized in our research. Firstly, we define $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ as a directed graph, where $\mathcal{E}$ denotes the collection of edges in graph $\mathcal{G}$, and $\mathcal{V}$ denotes the collection of nodes in graph $\mathcal{G}$. $\mathbf{E} = \{e_{uv} \mid \forall uv \in \mathcal{E}\}$ denotes the edge feature matrix, and $\mathbf{X} = \{x_i \mid i \in N\}$ denotes the node feature matrix. We use $N = |\mathcal{V}|$ to indicate the number of nodes. Let $\mathbf{D}$ be the diagonal degree matrix of $\mathbf{A}$, and $\mathbf{A}$ be the adjacency matrix of $\mathcal{G}$. The symmetric normalized adjacency matrix is denoted as $\mathbf{A}_{\text{sym}} = \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$, and the random walk matrix is denoted as $\mathbf{A}_{\text{rw}} = \mathbf{A}\mathbf{D}^{-1}$. Let $\hat{\mathbf{A}}$ be any transition matrix, including $\mathbf{A}_{\text{sym}}$, $\mathbf{A}_{\text{rw}}$, or other matrices representing message passing. We use $\mathbf{M}_{i:}$ and $\mathbf{M}_{j:}$ to represent the ith and jth rows of matrix $\mathbf{M}$, respectively. Additionally, let $[\![0, K]\!]$ denote the set $\{0, 1, \ldots, K\}$.

3.1. Multi-hop graph attention networks

Standard graph neural networks typically perform message passing and neighborhood aggregation once per layer. Recently, diffusion-based models and some graph neural networks have begun implementing multi-hop message passing within a single layer (Chien et al., 2021; Zhao et al., 2021). One of the most representative models in this context is the decoupled GCN (Wu et al., 2019), which alleviates the over-smoothing problem in GCNs caused by the coupling of neighborhood aggregation and feature transformation. Decoupled GCNs execute feature transformation and neighborhood aggregation separately. Each hop has its aggregation weights, and multiple hops are used to achieve effective message passing. However, this method also has a problem: when the number of hops becomes too large, it inevitably leads to the over-smoothing phenomenon. In recent work, there have been some attention-based methods that utilize multi-hop information. Chen et al. (2022) began using K-hop representation aggregation and employed a Transformer model to process K-step message propagation. Although this method performs well, it still relies on K-step propagation to gather multi-hop information prior to incorporating the attention mechanism, leading to the problem. Therefore, incorporating the attention mechanism into the propagation phase might be a better choice.

The global self-attention computation is performed by projecting the node features into three subspaces and calculating a weighted sum at each position. We use $\text{SA}(\cdot, \cdot, \cdot)$ to represent the global self-attention function, with the projected subspaces $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ represented as follows:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \mathbf{K} = \mathbf{X}\mathbf{W}_K, \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{1}$$

where $\mathbf{W}_Q \in \mathbb{R}^{d \times d_K}$, $\mathbf{W}_K \in \mathbb{R}^{d \times d_K}$, and $\mathbf{W}_V \in \mathbb{R}^{d \times d_V}$ are learnable transformation matrices. By performing the computation, we obtain the

new representation for the ith node as follows:

$$SA(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:} = \frac{\sum_{j=1}^{N} sim(\mathbf{Q}_{i:},\mathbf{K}_{j:})\mathbf{V}_{j:}}{\sum_{j=1}^{N} sim(\mathbf{Q}_{i:},\mathbf{K}_{j:})} \tag{2}$$

where $\mathrm{sim}(\cdot,\cdot) : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is a function that assesses the similarity between the query and the key, a standard computation method. Huang et al. (2024) used a more efficient algorithm, employing a message-passing mechanism to achieve multi-hop attention computation. They call this algorithm subtree attention, the same as the multi-hop attention we use here. Letting keys and values travel along the edges reduced the algorithm's complexity to linear time. Additionally, this approach also reduces space complexity. Traditional attention mechanisms involve storing the transition matrix $\widehat{\mathbf{A}}^k$, which can consume excessive GPU memory. The attention weight for the ith node in the kth layer is as follows:

$$MHA_k(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:} = \frac{\phi(\mathbf{Q}_{i:})\sum_{j=1}^{N}\widehat{\mathbf{A}}^k_{ij}\phi(\mathbf{K}_{j:})^T\mathbf{V}_{j:}}{\phi(\mathbf{Q}_{i:})\sum_{j=1}^{N}\widehat{\mathbf{A}}^k_{ij}\phi(\mathbf{K}_{j:})^T} \tag{3}$$

MHA_k refers to the attention weights between k-hop neighbors, where a feature mapping ϕ is used instead of $\mathrm{sim}(\cdot,\cdot)$. According to our current understanding, the algorithm $\phi(x) = \mathrm{elu}(x)+1$ proposed by Katharopoulos et al. (2020) performs comparably to *softmax* attention. Based on this algorithm, MHA attempts to integrate the message-passing mechanism into the full attention architecture, acting as a propagation module for keys and values. For more details on Eq. (3), please refer to Appendix B.

3.2. Edge-featured graph neural network

We know that in the field of network intrusion detection based on flow data, when using GNNs, a typical graph construction method is to form a graph based on the source IP, port, and destination IP, port, with other information as edge attributes. At this point, determining the type of request becomes one of the classification tasks: identifying whether a request is abnormal is a binary classification task; identifying the type of request is a multi-class classification task. In Edge-featured Graph Neural Networks, as far as we know, the E-GraphSAGE algorithm (Lo et al., 2022) is a well-known algorithm in network intrusion detection. Compared to the original GraphSAGE algorithm (Hamilton et al., 2017), the E-GraphSAGE proposed by Lo et al. differs in its input, output, and message propagation functions. Their propagation function incorporates edge features $\{\mathbf{e}_{uv}, \forall uv \in \mathcal{E}\}$ for message passing, which is the primary distinction from the original GraphSAGE. In network intrusion detection, where most information is concentrated on the edges, and node information is relatively sparse, their algorithm primarily targets edge features, specifically network flow data. To facilitate message passing, the algorithm initializes node features by uniformly assigning the feature vector $\mathbf{x}_v = \{1,\ldots,1\}$ to each node, ensuring that the dimensions of the node features match those of the edge features. Since edge features are used for neighborhood aggregation, they proposed a new aggregation function to aggregate edge features over k-layer neighborhood information. The formula is as follows:

$$\mathbf{h}^k_{\mathcal{N}(v)} = AGG_k\left(\left\{\mathbf{h}^{k-1}_u \parallel \mathbf{e}^{k-1}_{uv}, \forall u \in \mathcal{N}(v), uv \in \mathcal{E}\right\}\right) \tag{4}$$

where $\mathbf{e}^{k-1}_{uv}$ represents the edge feature uv from the neighborhood $\mathcal{N}(v)$ of node v at layer $k-1$, and $\mathbf{h}^{k-1}_u$ denotes the embedding of the neighboring node. By concatenating $\mathbf{e}^{k-1}_{uv}$ and $\mathbf{h}^{k-1}_u$, a new node representation is formed.

Subsequently, based on the k-hop edge features, the node embedding $\mathbf{h}_v$ at layer k is computed, with the neighbor node embeddings composed of the neighboring edge features. Therefore, the new feature representation $\mathbf{h}^k_{\mathcal{N}(v)}$ integrates the neighborhood feature information, which is then concatenated with the current node feature $\mathbf{h}^{k-1}_v$. Next, the weight parameter $\mathbf{W}^k$ is applied to the newly formed node embedding, and the result is passed through a nonlinear activation function to obtain the new node embedding. The specific formula is as follows:

$$\mathbf{h}^k_v = \sigma\left(\mathbf{W}^k \cdot CONCAT\left(\mathbf{h}^{k-1}_v, \mathbf{h}^k_{\mathcal{N}(v)}\right)\right) \tag{5}$$

The above describes the main logic of the E-GraphSAGE algorithm.

4. Methodology

In this section, we will discuss the data preprocessing steps, followed by an explanation of the motivation behind our proposed model. Then, we will detail the technical aspects of the model and demonstrate the relevant algorithms.

4.1. Pre-processing

Network data structure. In this study, we utilize network traffic (such as NetFlow) to construct the backend environment for a Network Intrusion Detection System (NIDS). These traffic records capture source and destination communications information, along with other metadata such as flow duration, the number of bytes, and the number of packets sent. As shown in Fig. 1, we treat the nodes formed by IP addresses and port numbers as unique entities in cyberspace, with requests between the source and destination nodes viewed as edges between these nodes. The request information serves as edge features, including the request duration, the number of packets in the request, and the number of packets in the response.

Feature extraction. We used all data flow fields (such as total bytes/flows/packets, incoming/outgoing traffic packet counts, incoming/outgoing traffic bytes, and duration) as edge features. Invalid data or NaN values were set to 0, and data exceeding the 32-bit range were discarded. Node features were extracted using target and count encoding to capture the ratio and frequency of nodes as source/destination endpoints across different label groups. Nguyen and Kashef (2023)'s work has indicated that these extracted node features benefit network intrusion detection tasks through this method. The *StandardScaler* method was applied to standardize both node and edge features.

4.2. Motivations

Owing to the constraints of traditional intrusion detection techniques, signature-based detection models are no longer suitable for complex and dynamic network attacks. In contrast, deep learning-based anomaly detection techniques can handle more sophisticated attack scenarios. Using graph neural networks in intrusion detection systems allows for capturing the topological relationships of network data, making it an excellent solution to these challenges. However, conventional GNN models have certain drawbacks, including over-smoothing in GCNs and the computational overhead in GATs. Current research has moved beyond simply applying GNNs, focusing instead on combining the strengths of GNNs and transformers to optimize existing models (Gonçalves and Zanchettin, 2024). Our study aims to explore the application of graph transformers in intrusion detection, alleviating potential issues such as over-smoothing and instability in detection performance associated with conventional GNN models.

4.3. The proposed EMA-IDS model

To address the issues mentioned above, we propose a new model called an Edge-featured Multi-hop Attention Graph Neural Network for Intrusion Detection System (EMA-IDS). EMA-IDS comprises three main components: a node encoding module, an edge encoding module, and an MLP module.

Fig. 2 illustrates the overall architecture of our model. First, the data is preprocessed and then fed into the model. The data flows through

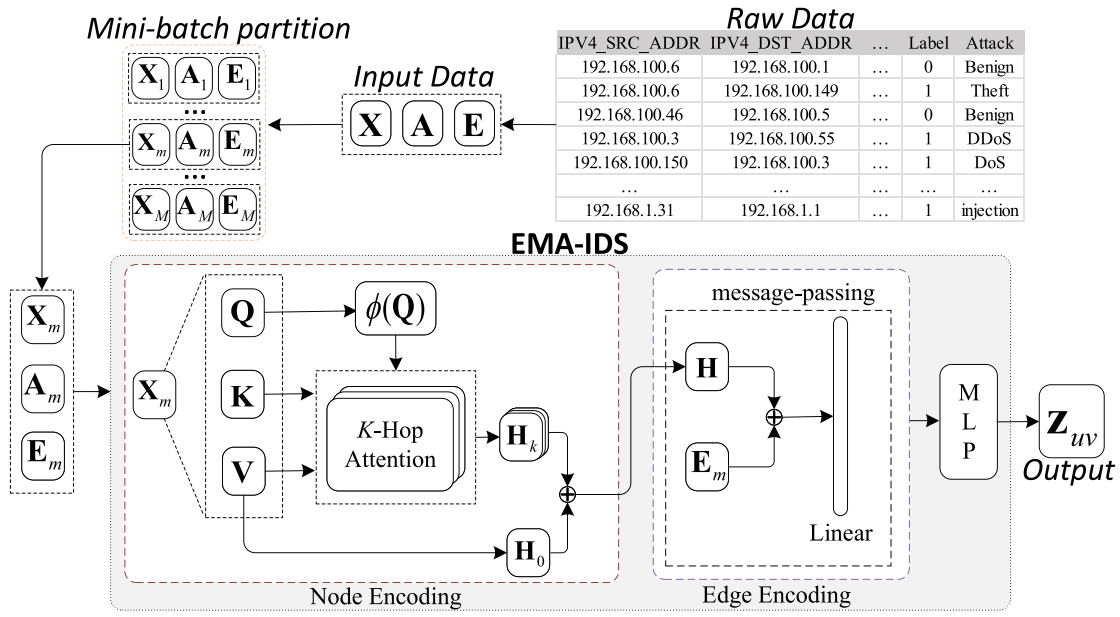

Fig. 2. Illustrates the main structure and data flow of the EMA-IDS model. The raw data is processed to obtain node features $\mathbf{X}$, the adjacency matrix $\mathbf{A}$, and edge features $\mathbf{E}$. We randomly sample data to form mini-batches of input subgraphs, each containing $\mathbf{X}_m$, $\mathbf{A}_m$, and $\mathbf{E}_m$. $\mathbf{X}_m$ and $\mathbf{A}_m$ are fed into the node encoding module, where $\mathbf{X}_m$ is projected to generate $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$. After performing multi-hop attention propagation on the subgraph, new hidden representations $\mathbf{H}$ are generated. $\mathbf{H}$, $\mathbf{A}_m$, and $\mathbf{E}_m$ are then input into the edge encoding module, where $\mathbf{H}$ is concatenated with the weighted $\mathbf{E}_m$ and undergoes a linear transformation during message passing. Finally, a shallow multi-layer perceptron (MLP) is applied to obtain the edge embeddings.

the node encoding module, the edge encoding module, and the MLP module, and finally, the loss is computed.

Node Encoding. This part mainly focuses on node features. We followed the work of Huang et al. (2024) for node encoding, and We observed that the linear outputs of $\mathbf{W}_Q$ and $\mathbf{W}_K$ can be kept minimal as long as the output of $\mathbf{W}_V$ remains appropriate for downstream tasks. This approach can significantly reduce the model's GPU memory usage without affecting classification accuracy by minimizing the linear outputs of $\mathbf{W}_Q$ and $\mathbf{W}_K$. Node Encoding consists of three steps: node feature transformation, multi-hop attention computation, and multi-hop attention aggregation. First, the node features are linearly transformed to an appropriate dimension and then projected into subspaces $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ using Eq. (1). We apply the feature mapping function ϕ proposed by Katharopoulos et al. (2020) to process the queries $\mathbf{Q}$ and keys $\mathbf{K}$.

Next, we compute the multi-hop attention. Note that in Eq. (3), there are two summations: $\sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^{k} \phi(\mathbf{K}_{j:})^T \mathbf{V}_{j:}$ and $\sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^{k} \phi(\mathbf{K}_{j:})^T$. These two summations can be viewed as a form of message passing. This means that we first compute $\phi(\mathbf{K}_{i:})$ and $\phi(\mathbf{K}_{i:})^T \mathbf{V}_{i:}$. Next, we perform k steps of message passing with $\phi(\mathbf{K}_{i:})$ and $\phi(\mathbf{K}_{i:})^T \mathbf{V}_{i:}$. Finally, we combine the node's own $\phi(\mathbf{Q}_{i:})$ with the aggregated keys and values $\sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^{k} \phi(\mathbf{K}_{j:})^T \mathbf{V}_{j:}$ and $\sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^{k} \phi(\mathbf{K}_{j:})^T$ to complete the computation of $MHA_k(\mathbf{Q}, \mathbf{K}, \mathbf{V})_{i:}$. Fig. 3 illustrates the computation process of multi-hop attention for a single node. We use $\mathbf{A}_{\text{rw}}$ as the transition matrix, enabling the propagation of keys and values along the edges. This process can be viewed as a random walk on the graph.

Then, all level results are aggregated using Eq. (6). The specific *AGGR* function can be sum, concat, attention-based readout (Chen et al., 2022), or GPR-like aggregation (Chien et al., 2021). Eq. (6) is as follows:

$$MHA(\mathbf{Q}, \mathbf{K}, \mathbf{V})_{i:} = AGG\left(\{\text{MHA}_k(\mathbf{Q}, \mathbf{K}, \mathbf{V})_{i:} \parallel k \in [\![0, K]\!]\}\right) \tag{6}$$

Edge Encoding. We input the embeddings processed by the node encoding module and the edge features into the edge encoding module. Before calculating with Eq. (8), we perform attention computation on the edge features. The features calculated with attention parameters

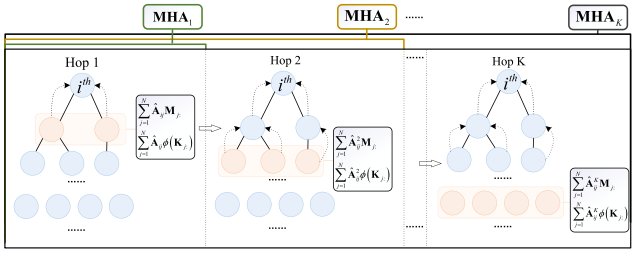

Fig. 3. Illustrates the computation process of K-hop attention for the ith node. We need to compute $\sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^{K} \mathbf{M}_{j:}$ and $\sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^{K} \phi(\mathbf{K}_{j:})$, where $\mathbf{M}_{j:} = \phi(\mathbf{K}_{j:})^T \mathbf{V}_{j:}$. After computing MHA_1, we can directly use the results of MHA_1 when computing MHA_2, thus avoiding redundant computations. This process continues until MHA_K, where each step utilizes the results from the previous $K-1$ computations. Therefore, the computation of $\{MHA_k\}_{k \in [\![1,K]\!]}$ is a nested process.

can better adjust the influence of different edges on the nodes. The edge feature attention computation is as follows:

$$\alpha^k = softmax(LeakyReLU(W_e^k \cdot e_{uv}^{k-1})) \tag{7}$$

By multiplying the edge feature e_{uv}^{k-1} with the weight parameter W_e^k, we obtain a weighted edge feature. Then, the edge feature is processed using the *LeakyReLU* activation function to obtain the attention coefficient α^k. Finally, the attention coefficient α^k is normalized using the *softmax* operation. The obtained attention coefficient is then multiplied by the edge feature, resulting in an edge feature weighted by the attention mechanism. This process better captures the important information on the edges. The specific formula is as follows:

$$h_{\mathcal{N}(v)}^k = AGG\left(\left\{h_u^{k-1} \parallel (e_{uv}^{k-1} \cdot \alpha^k), \forall u \in \mathcal{N}(v), uv \in \mathcal{E}\right\}\right) \tag{8}$$

where $h_{\mathcal{N}(v)}^k$ represents the aggregated information from the neighboring nodes. As described in Eq. (5), a new feature representation is obtained by merging the neighbor node features with the node's own features and applying an activation function.

MLP. The feature representations generated by the edge encoding need to be processed through a shallow MLP. In this case, we set the

MLP to have two layers. After the MLP, a specific *READOUT* function is applied to obtain the edge embeddings. The transformation of node embeddings to edge embeddings can be expressed as follows:

$$\mathbf{z}_{uv} = READOUT\left(\mathbf{z}_u, \mathbf{z}_v\right), uv \in \mathcal{E} \tag{9}$$

where $READOUT(\cdot,\cdot)$ can be any readout function, such as concatenation operator, L1, L2, or Hadamard. In our proposed method, we choose concatenation as the readout function.

Note that our edge encoding performs only one step of message passing, meaning $k = 1$ in the edge encoding module. We observed that a single integration of edge features and node features is sufficient to capture the spatial topology of the graph. algorithm 1 illustrates all the above processes and the training flow of our model.

Algorithm 1: The EMA-IDS algorithm

Input:

- A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
- Input node features $\mathbf{X} = \{x_i \mid i \in N\}$
- Input edge features $\mathbf{E} = \{e_{uv} \mid \forall uv \in \mathcal{E}\}$
- Parameters: multi-hop K

Output: Edge embeddings $\mathbf{Z} = \{z_{uv} \mid \forall uv \in \mathcal{E}\}$

for *each iteration* **do**

/* node encoding */

$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q,\ \mathbf{K} = \mathbf{X}\mathbf{W}_K,\ \mathbf{V} = \mathbf{X}\mathbf{W}_v$

$\phi(\mathbf{Q}) = elu(\mathbf{Q}) + 1,\ \phi(\mathbf{K}) = elu(\mathbf{K}) + 1$

foreach $k = 1, 2, \ldots, K$ **do**

$$MHA_k(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:} = \phi(\mathbf{Q}_{i:}) \sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^k \phi(\mathbf{K}_{j:})^T \mathbf{V}_{j:} / \phi(\mathbf{Q}_{i:}) \sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^k \phi(\mathbf{K}_{j:})^T$$

$$MHA(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:} = AGG\left(\{MHA_k(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:} \mid k \in [\![0, K]\!]\}\right)$$

end

/* edge encoding */

$\alpha^1 = softmax(LeakyReLU(W_e^1 \cdot e_{uv}^0))$

$h_{\mathcal{N}(v)}^1 = AGG\left(\{h_u^0 || (e_{uv}^0 \cdot \alpha^1), \forall u \in \mathcal{N}(v), uv \in \mathcal{E}\}\right)$

$\mathbf{h}_v^1 = \sigma\left(\mathbf{W}^1 \cdot CONCAT\left(\mathbf{h}_v^0, \mathbf{h}_{\mathcal{N}(v)}^1\right)\right)$

/* edge embeddings readout */

$\mathbf{z}_{uv} = READOUT\left(\mathbf{z}_u, \mathbf{z}_v\right), uv \in \mathcal{E}$

end

5. Experiments

In this section, we evaluate our proposed EMA-IDS model through experiments on four benchmark datasets. We begin by outlining the experimental setup. Next, we compare our approach against five benchmark models to assess the performance of EMA-IDS in both binary and multi-class classification tasks. Finally, we analyze the results of our method on these datasets.

5.1. Datasets

We employ the NetFlow dataset (Sarhan et al., 2021, 2022), which comprises four benchmark subsets: NF-CSE-CIC-IDS2018-v2, NF-UNSW-NB15-v2, NF-BoT-IoT, and NF-ToN-IoT. These subsets are derived from the original CSE-CIC-IDS2018, UNSW-NB15, BoT-IoT, and ToN-IoT datasets, respectively. Each subset offers two types of labels for prediction: one type with two categories to indicate whether the traffic is normal or under attack, and the other with multiple categories to represent different attack types.

Table 1 shows the data categories contained in the four datasets. NF-BoT-IoT is a relatively small dataset with five types, including 586,241 (97.69%) attack data. NF-ToN-IoT contains more data than NF-BoT-IoT, with ten types. NF-UNSW-NB15-v2 is an updated version of NF-UNSW-NB15 with a larger overall data volume than the former. NF-CSE-CIC-IDS2018-v2 is the largest dataset among the four, with a total data volume of 18,893,708. The large-scale data volume also better evaluates the model's performance when dealing with vast data.

Table 1
Classes and samples of four datasets.

Dataset	Classes	Samples
NF-BoT-IoT	Benign	13,859
	DDoS	56,844
	Theft	1 909
	DoS	56,833
	Reconnaissance	470,655
NF-ToN-IoT	xss	99,944
	password	156,299
	ransomware	142
	scanning	21,467
	mitm	1 295
	Benign	270,279
	backdoor	17,247
	injection	468,539
	ddos	326,345
	dos	17,717
NF-CSE-CIC-IDS2018-v2	Benign	16,635,567
	Bot	143,097
	BruteForce	120,912
	DDOS	1,390,270
	DoS	483,999
	Infiltration	116,361
	Web Attacks	3 502
NF-UNSW-NB15-v2	Analysis	2 299
	Backdoor	2 169
	Benign	2,295,222
	DoS	5 794
	Exploits	31,551
	Worms	164
	Fuzzers	22,310
	Reconnaissance	12,779
	Shellcode	1 427
	Generic	16,560

After preprocessing the four datasets, we obtained eight preprocessed files corresponding to binary and multi-class classification tasks for each of the four datasets. Table 2 presents the basic graph data information for these eight preprocessed files, including the number of nodes, edges, node and edge feature dimensions, and labels. Additionally, we provided scatter plots of node degrees for the graphs constructed from the four datasets, as shown in Fig. 4.

5.2. Baselines

In this experiment, we will evaluate our proposed method against five baseline models, which are considered state-of-the-art in network intrusion detection, utilizing both deep learning (DL) and machine learning (ML) techniques.

- KNN (Zhang et al., 2017) is a widely adopted ML algorithm for anomaly detection. Its main advantages include simplicity and ease of use, no training process required, and applicability to various data types.
- GAT (Velickovic et al., 2017) comprises attention mechanism-based graph neural networks. It allows nodes to dynamically adjust their attention to other nodes during the message-passing process, thereby better capturing local and global information within the graph structure.
- E-GraphSAGE (Lo et al., 2022) is a GNN-based approach that alters the traditional message-passing mechanism by relying solely on edge features instead of node features. This method has demonstrated superior performance compared to other machine learning and deep learning techniques and is recognized as one of the pioneering approaches to utilize graph neural networks for network intrusion detection. However, E-GraphSAGE cannot run

Table 2
Information of eight graphs derived from the four datasets.

Dataset	NF-BoT-IoT		NF-ToN-IoT		NF-CSE-CIC-IDS2018-v2		NF-UNSW-NB15-v2	
Type	Binary	Multi-cls.	Binary	Multi-cls.	Binary	Multi-cls.	Binary	Multi-cls.
Nodes	77,177	77,177	169,562	169,562	9,686,040	9,686,040	1,090,451	1,090,451
Edges	600,100	600,100	1,379,274	1,379,274	18,893,708	18,893,708	2,390,275	2,390,275
Node Feat. Dim.	8	32	8	72	8	112	8	72
Edge Feat. Dim.	8	8	8	8	39	39	39	39
Labels	2	5	2	10	2	15	2	10

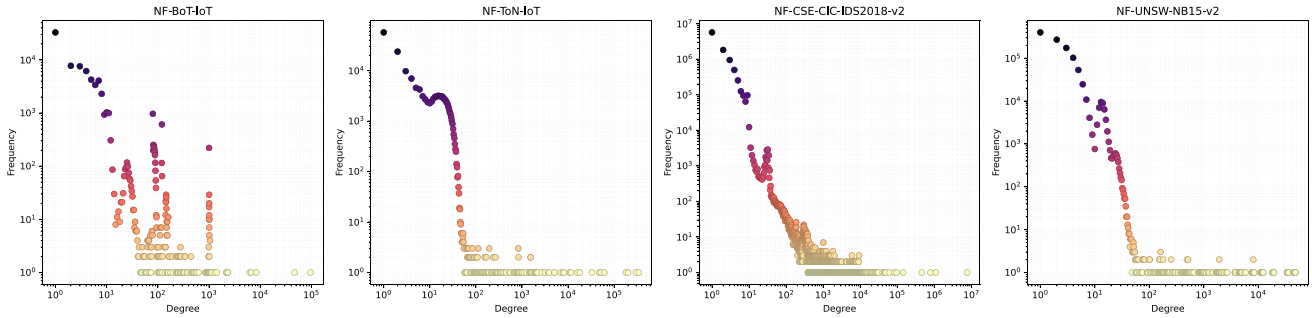

Fig. 4. Scatter plots of node degrees based on the graphs constructed from the four datasets.

on large-scale data because it attempts to input all data into the GPU simultaneously, exceeding GPU memory limits. Therefore, we retained the E-GraphSAGE model's code but used mini-batch training for data input.

- Anomal-E (Caville et al., 2022) is an enhanced method built upon E-GraphSAGE, integrating a self-supervised technique (DGI) to maximize the local–global mutual information of the graph by randomly shuffling edge features. After training, the encoder produces edge embeddings used as inputs for standard anomaly detection methods (e.g., HBOS, CBLOF, PCA, and IF). However, because its detectors depend on unsupervised learning algorithms, a direct evaluation against our proposed method is not feasible. Thus, we utilize the refined E-GraphSAGE encoder from the Anomal-E framework to generate edge embeddings, which are then input into an XGBoost (Chen and Guestrin, 2016) classifier to detect attacks. Similarly, mini-batch training is used for data input.
- TS-IDS (Nguyen and Kashef, 2023) is a recent and innovative method in intrusion detection. It improves data classification accuracy by employing a high-throughput self-supervised approach to focus on anomalous nodes, assisting the model in identifying abnormal nodes. As a GNN-based method, it is a valuable reference in intrusion detection.

We also propose variants of EMA-IDS, named EMA-IDS-M and EMA-IDS-E. These models contain only the node encoding module and only the edge encoding module, respectively. By evaluating these two models, we can better assess the performance of the EMA-IDS model across various datasets.

5.3. Parameter settings

Since the original method by the authors inherently supports mini-batch training, TS-IDS[1] will be set up strictly as the authors implemented it. For the baselines Anomal-E[2] and E-GraphSAGE,[3] we will adopt the original model architecture and hyperparameters provided by the authors. Due to GPU memory constraints, the input data will be processed in mini-batches instead of all at once, allowing for efficient training on large-scale data. The input features for KNN will be the same as those used for E-GraphSAGE and Anomal-E. The setup for the GAT method will be the same as our proposed method.

Table 3
Partitions of four datasets.

Dataset	No. data	Train	Val	Test
NF-BoT-IoT	600,100	360,060	120,020	120,020
NF-ToN-IoT	1,379,274	827,564	275,855	275,855
NF-CSE-CIC-IDS2018-v2	18,893,708	11,336,226	3,778,741	3,778,741
NF-UNSW-NB15-v2	2,390,275	1,434,165	478,055	478,055

We will use cross-validation to evaluate the models' performance and stability. In this experiment, we choose to perform five-fold cross-validation. The dataset is divided into training, validation, and test sets for each fold, making up 60%, 20%, and 20% of the entire dataset, respectively. Table 3 shows the sizes of these sets for each dataset after division.

We implemented our proposed model and training process using Python (Van Rossum and Drake, 1995), PyTorch (Paszke et al., 2019), PyTorch-Geometric (Fey and Lenssen, 2019), DGL (Wang, 2019), and PyTorch-Lightning.[4] We conducted the experiments using an NVIDIA Tesla V100 GPU with 16 GB of memory. Additionally, due to its large scale, we used a separate NVIDIA Tesla V100 GPU with 32 GB of memory for tasks on the NF-CSE-CIC-IDS2018-v2 dataset. We set the learning rate to a value within the range $[10^{-3}, 10^{-2}]$ and utilized early stopping based on validation set accuracy during model training. Additional hyperparameters are detailed in Table 4. It is also worth mentioning that feature preprocessing was performed for each fold based on the training and validation datasets before being applied to the entire dataset.

The work related to this paper is available at https://github.com/topsecd/EMA-IDS.

5.4. Evaluation metrics

We used the trained models to evaluate performance on the test set. The following evaluation metrics were employed, where TN, FN, FP, and TP represent True Negative, False Negative, False Positive, and True Positive, respectively.

[1] https://github.com/hoangntc/TS-IDS.
[2] https://github.com/waimorris/Anomal-E.
[3] https://github.com/waimorris/E-GraphSAGE.
[4] https://lightning.ai.

Table 4
The settings for the hyperparameters in our experiment.

Hyperparameter	Value
Optimizer	*Adam*
Loss function	*CE*
Dropout	0.2
Batch size	30,000
Learning rate	$[10^{-3}, 10^{-2}]$
Max epochs	500
No. multi-hop	[1–3]
No. hidden	64
No. d_K	[1,2,4]

- Accuracy(ACC):

$$ACC = \frac{TP + TN}{TP + FP + TN + FN} \tag{10}$$

- Area Under the Curve (AUC): includes the False Positive Rate (FPR) and the True Positive Rate (TPR).

$$TPR = \frac{TP}{TP + FN} \tag{11}$$

$$FPR = \frac{FP}{FP + TN} \tag{12}$$

- F1-score: is composed of Recall and Precision.

$$Precision = \frac{TP}{TP + FP} \tag{13}$$

$$Recall = \frac{TP}{FN + TP} \tag{14}$$

$$F1 = 2 \times \frac{Recall \times Precision}{Recall + Precision} \tag{15}$$

5.5. Experimental results

5.5.1. Binary classification results

We used ACC, AUC, weighted F1, macro recall, and macro precision as evaluation metrics for the binary classification task. Table 5 presents the outcomes of each comparison model on the four benchmark datasets.

From the experimental data, our model performs exceptionally well in most metrics. However, on the NF-CSE-CIC-IDS2018-v2 dataset, the AUC, Precision, and Recall metrics are slightly weaker compared to other models, which indicates that while the original E-GraphSAGE and AnomalE models could not run on the large NF-CSE-CIC-IDS2018-v2 dataset, their performance is still outstanding when using mini-batch inputs. On the NF-UNSW-NB15-v2 dataset, our model also achieved the best results in ACC and F1 metrics. Interestingly, KNN had the highest Precision, demonstrating that traditional ML methods still perform well in anomaly detection.

GAT and TS-IDS, in contrast, showed moderate performance in this binary classification task. Comparing their results, we found that GAT slightly outperformed TS-IDS in some metrics. Upon analyzing the TS-IDS code from preprocessing to model structure, we discovered that TS-IDS includes an auxiliary label extraction process. These auxiliary labels are used to learn whether a node is an anomalous endpoint. The extracted auxiliary labels are not always accurate compared to the true labels, potentially leading to the learning of incorrect information. Nevertheless, using high-throughput attention on endpoints is a novel approach, and it will still perform excellently on large-scale data in real-world scenarios. EMA-IDS-E shows slightly weaker performance than EMA-IDS-M on the NF-UNSW-NB15-v2 dataset, but their binary classification performance is similar on the other three datasets.

5.5.2. Multi-class classification results

For the multi-class classification experiment, we use weighted recall and weighted F1 to evaluate the model's performance. Table 6 presents the results of the multi-class classification experiments. For more details on our model's performance across multiple attack types, please refer to Appendix A.

From the experimental data, our model achieved the best metrics across all datasets. For traditional machine learning methods, KNN performed very well on most datasets except for NF-ToN-IoT, where its performance was slightly weaker. KNN's performance demonstrates that traditional ML methods can still be sufficient in some scenarios if extreme performance is not required.

For EGraphSAGE and AnomalE, EGraphSAGE outperformed AnomalE on the NF-BoT-IoT and NF-UNSW-NB15-v2 datasets, while the opposite was true for the other two datasets. Since AnomalE is based on E-GraphSAGE, their similar performance is expected.

TS-IDS performed better than GAT on all four benchmark datasets, supporting the authors' claim that their proposed approach improves the model's detection capability. Our proposed EMA-IDS method outperformed TS-IDS by 3% on the NF-BoT-IoT dataset and 4% on the NF-ToN-IoT dataset, indicating that multi-hop attention positively impacts multi-class classification tasks.

On the large-scale NF-CSE-CIC-IDS2018-v2 dataset, all models performed similarly, which suggests that the abundance of information can benefit the learning process with large datasets, making the choice of ML method less critical.

5.5.3. Ablation study

To evaluate the impact of multi-hop attention on the model, we proposed two variants of the EMA-IDS model: EMA-IDS-M and EMA-IDS-E. EMA-IDS-M includes the node encoding module but not the edge encoding module, while EMA-IDS-E includes only the edge encoding module. We observed that EMA-IDS-E outperforms EMA-IDS-M in most scenarios. Still, EMA-IDS-M performed slightly better in the multi-class classification on the NF-ToN-IoT dataset and the binary classifications on the NF-CSE-CIC-IDS2018-v2 and NF-UNSW-NB15-v2 datasets. The full EMA-IDS model, which incorporates both modules, outperforms these variant models, indicating that the contributions of both modules positively impact the final model. For detailed results, please refer to Tables 5 and 6.

In the following discussion, K represents the value of multi-hop attention propagation, and L represents the number of convolution layers.

Study on the comparison of K-hop propagation and L-layer convolution. Since traditional GCN mainly processes node features and does not handle edge features, we used our EMA-IDS-E for experimental observation to utilize edge features effectively. In this study, we executed K-hop attention propagation on EMA-IDS, while performing L-layer convolution on EMA-IDS-E, with K and L ranging from 1 to 10. We selected the weighted F1 as the evaluation metric. The four datasets' experimental results for multi-class tasks are shown in the line chart of Fig. 5. Except for NF-CSE-CIC-IDS2018-v2, where performance fluctuates significantly in the latter half, EMA-IDS demonstrates stable performance in other cases. These observations provide strong evidence that multi-hop attention propagation offers robustness and efficiency. We did not observe a drastic performance drop for EMA-IDS-E after multiple convolutional layers. Instead, the performance gradually decreased as the number of layers increased. Upon analyzing the model's code, unlike traditional node classification tasks that only involve node features, when a dataset includes edge features, the hidden representations are combined with edge features during each round of message passing and aggregation. This method of handling message passing and aggregation might be the reason for this phenomenon and represents a key difference between edge classification tasks in network intrusion detection and other tasks.

Table 5
Experimental results of binary classification. Underlined for best performance.

Dataset	Metric	KNN	EGraphSAGE	AnomalE	GAT	TS-IDS	EMA-IDS	EMA-IDS-M	EMA-IDS-E
NF-BoT-IoT	ACC	0.9478	0.9102	0.9144	0.9205	0.9151	0.9522	0.9297	0.9265
	AUC	0.8542	0.9415	0.9362	0.9518	0.9515	0.9653	0.9245	0.9526
	F1	0.9595	0.9376	0.9401	0.9439	0.9406	0.9636	0.9492	0.9475
	Recall	0.8542	0.9415	0.9362	0.9518	0.9515	0.9653	0.9245	0.9526
	Precision	0.6333	0.6006	0.6081	0.6115	0.6065	0.6613	0.6178	0.6180
NF-ToN-IoT	ACC	0.9427	0.9799	0.9849	0.9888	0.9923	0.9988	0.9933	0.9938
	AUC	0.9578	0.9755	0.9887	0.9928	0.9937	0.9976	0.9943	0.9954
	F1	0.9450	0.9800	0.9851	0.9889	0.9923	0.9988	0.9934	0.9938
	Recall	0.9578	0.9755	0.9887	0.9928	0.9937	0.9976	0.9943	0.9954
	Precision	0.8883	0.9620	0.9657	0.9733	0.9824	0.9985	0.9850	0.9855
NF-CSE-CIC-IDS2018-v2	ACC	0.9956	0.9948	0.9953	0.9911	0.9915	0.9958	0.9954	0.9950
	AUC	0.9828	0.9791	0.9933	0.9769	0.9769	0.9915	0.9918	0.9917
	F1	0.9955	0.9948	0.9953	0.9911	0.9915	0.9958	0.9954	0.9951
	Recall	0.9828	0.9791	0.9933	0.9769	0.9769	0.9915	0.9918	0.9917
	Precision	0.9960	0.9963	0.9847	0.9806	0.9827	0.9886	0.9865	0.9849
NF-UNSW-NB15-v2	ACC	0.9956	0.9947	0.9933	0.9908	0.9888	0.9956	0.9916	0.9905
	AUC	0.9963	0.9964	0.9948	0.9804	0.9939	0.9973	0.9952	0.9919
	F1	0.9957	0.9949	0.9936	0.9911	0.9894	0.9957	0.9920	0.9910
	Recall	0.9963	0.9964	0.9948	0.9804	0.9939	0.9973	0.9952	0.9919
	Precision	0.9511	0.9422	0.9294	0.9140	0.9301	0.9501	0.9129	0.9070

Table 6
Experimental results of multi-class classification. Underlined for best performance.

Model	NF-BoT-IoT		NF-ToN-IoT		NF-CSE-CIC-IDS2018-v2		NF-UNSW-NB15-v2	
	Recall	F1	Recall	F1	Recall	F1	Recall	F1
KNN	0.8353	0.8389	0.5484	0.4797	0.9956	0.9945	0.9871	0.9870
EGraphSAGE	0.8223	0.7931	0.5748	0.5064	0.9755	0.9668	0.9834	0.9831
AnomalE	0.7819	0.7840	0.6608	0.6466	0.9961	0.9955	0.9762	0.9728
GAT	0.7860	0.7739	0.6543	0.6322	0.9971	0.9969	0.9710	0.9708
TS-IDS	0.8153	0.8134	0.6699	0.6303	0.9972	0.9970	0.9730	0.9670
EMA-IDS	0.8397	0.8519	0.7158	0.6741	0.9974	0.9972	0.9874	0.9875
EMA-IDS-M	0.8257	0.8022	0.6839	0.6688	0.9969	0.9967	0.9801	0.9783
EMA-IDS-E	0.8298	0.8304	0.6505	0.6323	0.9971	0.9969	0.9827	0.9812

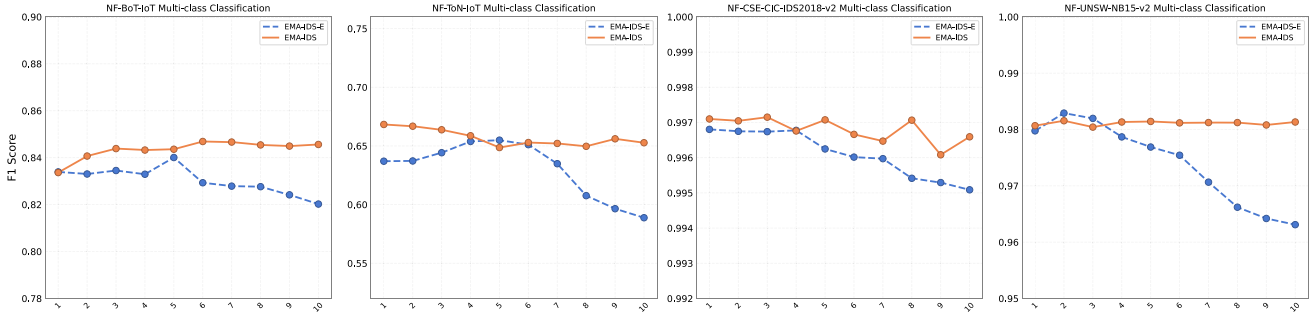

Fig. 5. The multi-class classification results after EMA-IDS and EMA-IDS-E perform K-hop propagation or L-layer convolution, respectively. The x-axis of each line chart represents K-hop propagation and L-layer convolution, while the y-axis represents the evaluation metric, F1 Score.

Study on EMA-IDS with L-layer convolution. By default, the edge encoding module contains only one convolution layer. To further explore the optimal model structure, we conducted additional experiments. We set the K-hop propagation value for the node encoding module, K, to 3 and performed extensive experiments with L ranging from 1 to 10 layers. As shown in Fig. 6 of the multi-class classification results across all datasets, We observed a slight improvement in performance with multiple convolution layers on the NF-CSE-CIC-IDS2018-v2 and NF-UNSW-NB15-v2 datasets, although there was a downward trend in performance on the NF-BoT-IoT and NF-ToN-IoT datasets.

From these comparisons, we found that varying the multi-hop values offers better stability compared to stacking convolution layers. Based on our best practice, we recommend setting K to 3 and L to 1.

5.5.4. Time complexity

Our proposed model consists of three components: the node encoding module, the edge encoding module, and the MLP. Since the time complexity of the MLP is mainly related to the parameter "No. Hidden", the analysis of our model's time complexity will focus primarily on the node encoding module and the edge encoding module.

For the node encoding module, the computation of MHA_k is divided into three steps. Firstly, it requires computing $\phi(\mathbf{K}_{i:})$ and $\phi(\mathbf{K}_{i:})^T\mathbf{V}_{i:}$, with time complexities of $\mathcal{O}(Nd_k)$ and $\mathcal{O}(Nd_kd_v)$, respectively. Then, for MHA_k, there are k propagations, and the time complexities for propagating $\phi(\mathbf{K}_{i:})$ and $\phi(\mathbf{K}_{i:})^T\mathbf{V}_{i:}$ once are $\mathcal{O}(d_k)$ and $\mathcal{O}(d_kd_v)$, respectively. Considering the propagation occurs along edges and is repeated k times, the total time complexity for this step

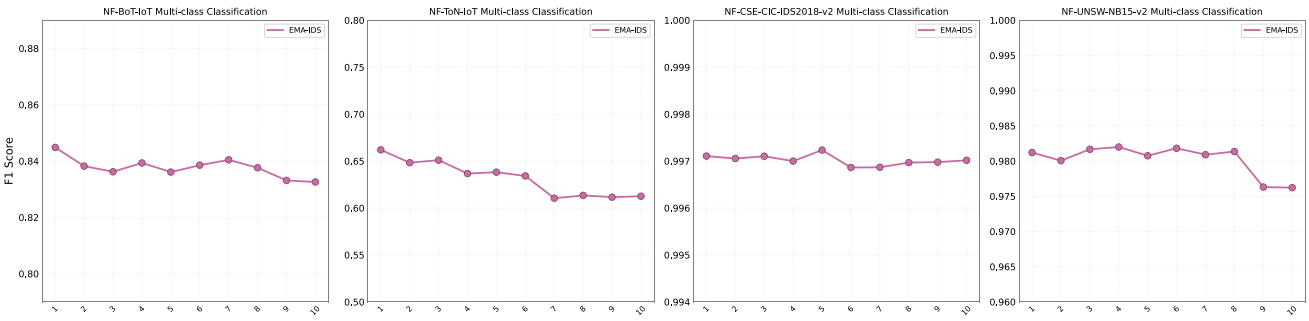

Fig. 6. The multi-class classification results after EMA-IDS performs L-layer convolution. The x-axis of each line chart represents L-layer convolution, while the y-axis represents the evaluation metric, F1 Score.

Table 7
Comparison of running time between EMA-IDS and baseline methods. Underline indicates the shortest runtime.

Running time	Model	NF-BoT-IoT		NF-ToN-IoT		NF-CSE-CIC-IDS2018-v2		NF-UNSW-NB15-v2	
		Binary	Multi-class	Binary	Multi-class	Binary	Multi-class	Binary	Multi-class
Training time (min)	EGraphSAGE	1.1806	1.3281	1.6919	2.5203	16.6455	92.1796	12.3458	32.3921
	AnomalE	1.9683	1.7489	3.1921	6.4984	91.5806	140.2054	9.9577	23.3067
	GAT	18.4217	27.6757	5.9037	17.3592	205.8706	311.6746	49.9093	59.9794
	TS-IDS	8.5648	9.7312	7.7155	7.3297	199.6631	282.9110	39.4220	53.1973
	EMA-IDS	3.0300	4.5481	6.0614	3.4652	14.1144	72.1318	11.9954	25.3894
Epoch time (s)	EGraphSAGE	0.6381	0.7378	2.4170	4.2005	23.7793	70.9074	4.7484	12.3008
	AnomalE	1.1466	1.2492	2.8165	4.6417	25.2057	77.1773	4.4256	11.1872
	GAT	4.4749	4.6908	7.2290	8.1371	119.9246	207.7831	20.7956	20.3320
	TS-IDS	3.4958	3.6492	5.3210	6.1941	108.9071	209.5637	16.6572	21.5665
	EMA-IDS	1.2201	1.3509	2.6546	4.4237	22.2858	65.5743	5.2154	11.9013

is $\mathcal{O}(k|\mathcal{E}|d_k + k|\mathcal{E}|d_k d_v)$. Finally, for each node, it needs to compute $\sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^k \ \phi(\mathbf{K}_{j:})^T \mathbf{V}_{j:}$ and $\sum_{j=1}^{N} \hat{\mathbf{A}}_{ij}^k \ \phi(\mathbf{K}_{j:})^T$, with time complexities of $\mathcal{O}(Nd_k d_v)$ and $\mathcal{O}(Nd_k)$, respectively. Thus, the complexity of computing MHA_k is $\mathcal{O}(2Nd_k + 2Nd_k d_v + k|\mathcal{E}|d_k + k|\mathcal{E}|d_k d_v)$.

Next, analyzing the time complexity of MHA, when the height of the root subtree is K, the computation of $\{MHA_i\}_{i\in[\![1,K]\!]}$ is a nested process, so its time complexity can be represented as $\mathcal{O}((K+1)Nd_k + (K+1)Nd_k d_v + K|\mathcal{E}|d_k + K|\mathcal{E}|d_k d_v)$. We can consider the time complexity of the node embedding module to be $\mathcal{O}(K|\mathcal{E}|d_k d_v)$.

For the edge encoding module, it is much simpler compared to the node encoding module. The primary task is to combine edge features with node features, with a time complexity of $\mathcal{O}(|\mathcal{E}|)$.

Consequently, our method has a time complexity of $\mathcal{O}(K|\mathcal{E}|d_k d_v)$.

5.5.5. Running time

In this subsection, we report the training time (in minutes) and the average time per epoch (in seconds) of our proposed method compared to baseline methods across different tasks. Since the time consumption for the KNN method mainly occurs during the prediction phase, we compare EMA-IDS with other deep learning methods. Many factors influence training duration, so we standardized these factors (e.g., batch size, learning rate, early stopping configuration) to ensure objectivity and fairness. As seen in Table 7, EGraphSAGE performs exceptionally well in most scenarios. Although EMA-IDS is not the fastest in some scenarios, its training speed is close to the best results. In contrast, GAT and TS-IDS take longer to train, especially on the NF-CSE-CIC-IDS2018-v2 dataset, where the time consumption is higher than other methods. This comparative experiment demonstrates that our method is relatively efficient.

6. Conclusion

This study investigated the use of graph neural networks (GNNs) for network intrusion detection. We introduced the EMA-IDS model, which better captures network topology information. This model combines node and edge features, making better use of the data characteristics. We implemented a node encoding module with multi-hop attention and an edge encoding module to compute edge attention, reducing overfitting and enhancing detection performance. From the comparative experiments in this paper, we observed that our proposed method outperforms existing ML and GNN methods.

Despite demonstrating some improvements, our model still has room for enhancement. Therefore, exploring capturing network data features in real-world scenarios is worthwhile.

The first limitation is that the model is divided into node encoding and edge encoding modules. Future work could focus on directly integrating edge features into the node module, making the method more efficient and reducing time and space complexity.

The second point is the rise of large language models (LLMs), which offer more options for network intrusion detection. Combining LLMs with the approach could improve anomaly detection in the future.

Additionally, this study did not address the issues of data imbalance and the temporality of attacks. These factors are crucial in intrusion detection. With the increasing prevalence of APT attacks, the context of these attacks should receive more attention. Research in this area would be precious.

CRediT authorship contribution statement

Ping Deng: Writing – original draft, Software, Methodology. **Yong Huang:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

links to the public data is included in the manuscript.

Table A.1
Multi-class classification results for NF-BoT-IoT.

Class name	No. samples	Precision	Recall	F1
DDoS	56,844	0.4654	0.4348	0.4496
DoS	56,833	0.3413	0.5498	0.4211
Benign	13,859	0.9224	0.5234	0.6679
Reconnaissance	470,655	0.9822	0.9323	0.9566
Theft	1,909	0.9833	0.6546	0.7860
Weighted average		**0.8725**	**0.8397**	**0.8519**

Table A.2
Multi-class classification results for NF-ToN-IoT.

Class name	No. samples	Precision	Recall	F1
backdoor	17,247	0.9772	0.9895	0.9833
Benign	270,279	0.9984	0.9891	0.9937
ddos	326,345	0.8733	0.7269	0.7934
ransomware	142	0.2857	0.0833	0.1290
dos	17,717	0.2535	0.0098	0.0190
password	156,299	0.3228	0.2425	0.2769
mitm	1,295	0.5303	0.1346	0.2147
injection	468,539	0.6066	0.9145	0.7294
scanning	21,467	0.0000	0.0000	0.0000
xss	99,944	0.7500	0.0004	0.0009
Weighted average		**0.7154**	**0.7158**	**0.6741**

Table A.3
Multi-class classification results for NF-CSE-CIC-IDS2018-v2.

Class name	No. samples	Precision	Recall	F1
Brute Force -XSS	927	0.9677	0.1840	0.3093
Brute Force -Web	2,143	0.9636	0.1235	0.2190
Bot	143,097	0.9992	1.0000	0.9996
Benign	16,635,567	0.9978	0.9994	0.9986
DoS attacks-Slowloris	9,512	0.9908	0.9702	0.9804
DoS attacks-SlowHTTPTest	14,116	0.9993	1.0000	0.9997
DoS attacks-Hulk	432,648	0.9999	1.0000	0.9999
DoS attacks-GoldenEye	27,723	0.9856	0.9957	0.9906
DDoS attacks-LOIC-HTTP	307,300	0.9997	0.9997	0.9997
DDOS attack-LOIC-UDP	2,112	1.0000	0.9746	0.9871
DDOS attack-HOIC	1,080,858	0.9973	1.0000	0.9986
Infilteration	116,361	0.8967	0.6866	0.7777
SQL Injection	432	1.0000	0.0714	0.1333
SSH-Bruteforce	94,979	0.9999	0.9996	0.9998
FTP-BruteForce	25,933	0.9975	1.0000	0.9988
Weighted average		**0.9972**	**0.9974**	**0.9972**

Table A.4
Multi-class classification results for NF-UNSW-NB15-v2.

Class name	No. samples	Precision	Recall	F1
Analysis	2,299	0.1631	0.7903	0.2703
Backdoor	2,169	0.5106	0.0543	0.0982
Benign	2,295,222	0.9975	0.9984	0.9979
Worms	164	0.0000	0.0000	0.0000
Shellcode	1,427	0.8604	0.6367	0.7318
Reconnaissance	12,779	0.9042	0.7685	0.8309
Generic	16,560	0.9350	0.7446	0.8290
Fuzzers	22,310	0.7986	0.7512	0.7742
Exploits	31,551	0.7887	0.8265	0.8072
DoS	5,794	0.4152	0.1199	0.1860
Weighted average		**0.9891**	**0.9874**	**0.9875**

Appendix A. Classification results of attack types

See Tables A.1–A.4.

Appendix B. Formula supplement

As shown in Eq. (2), computing $SA(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:}$ requires evaluating the similarity between queries and keys, which involves using a conventional softmax function as follows:

$$sim(\mathbf{Q}_{i:},\mathbf{K}_{j:}) = \exp\left(\frac{\mathbf{Q}_{i:}\mathbf{K}_{j:}^{T}}{\sqrt{d_K}}\right) \tag{B.1}$$

Recent research has proposed alternatives to this algorithm to reduce time complexity, such as Kernelized Softmax (Huang et al., 2024; Katharopoulos et al., 2020). By replacing the traditional similarity computation algorithm, we can rewrite $sim(\cdot,\cdot)$ as $\text{sim}(\mathbf{Q}_{i:},\mathbf{K}_{j:}) = \phi(\mathbf{Q}_{i:})\phi(\mathbf{K}_{j:})^{T}$, where ϕ represents the feature map. Thus, Eq. (2) can be rewritten as follows:

$$SA(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:} = \frac{\sum_{j=1}^{N}\phi(\mathbf{Q}_{i:})\phi(\mathbf{K}_{j:})^{T}\mathbf{V}_{j:}}{\sum_{j=1}^{N}\phi(\mathbf{Q}_{i:})\phi(\mathbf{K}_{j:})^{T}} \tag{B.2}$$

To continue simplifying the formula, we obtain the following:

$$SA(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:} = \frac{\phi(\mathbf{Q}_{i:})\sum_{j=1}^{N}\phi(\mathbf{K}_{j:})^{T}\mathbf{V}_{j:}}{\phi(\mathbf{Q}_{i:})\sum_{j=1}^{N}\phi(\mathbf{K}_{j:})^{T}} \tag{B.3}$$

When calculating $MHA_k(\mathbf{Q},\mathbf{K},\mathbf{V})_{i:}$, the transition matrix $\hat{\mathbf{A}}$ is also involved in balancing and normalizing the weights of neighboring nodes, which leads to Eq. (3). Each hop of attention propagation essentially corresponds to message passing on the graph, where the values $\sum_{j=1}^{N}\hat{\mathbf{A}}_{ij}^{k}\phi(\mathbf{K}_{j:})^{T}\mathbf{V}_{j:}$ and $\sum_{j=1}^{N}\hat{\mathbf{A}}_{ij}^{k}\phi(\mathbf{K}_{j:})^{T}$ are computed. Finally, the attention from each hop is aggregated through Eq. (6) to obtain the final hidden representations.

References

Al-Yaseen, W.L., 2019. Improving intrusion detection system by developing feature selection model based on firefly algorithm and support vector machine. IAENG Int. J. Comput. Sci. 46 (4), 534–540.

Balyan, A.K., Ahuja, S., Lilhore, U.K., Sharma, S.K., Manoharan, P., Algarni, A.D., Elmannai, H., Raahemifar, K., 2022. A hybrid intrusion detection model using ega-pso and improved random forest method. Sensors 22 (16), 5986.

Bayoudh, K., 2023. A survey of multimodal hybrid deep learning for computer vision: Architectures, applications, trends, and challenges. Inf. Fusion 102217.

Caville, E., Lo, W.W., Layeghy, S., Portmann, M., 2022. Anomal-E: A self-supervised network intrusion detection system based on graph neural networks. Knowl.-Based Syst. 258, 110030.

Chen, J., Gao, K., Li, G., He, K., 2022. Nagphormer: Neighborhood aggregation graph transformer for node classification in large graphs. CoRR abs/2206.04910.

Chen, T., Guestrin, C., 2016. Xgboost: A scalable tree boosting system. In: Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining. pp. 785–794.

Chen, D., Lin, Y., Li, W., Li, P., Zhou, J., Sun, X., 2020. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 34, pp. 3438–3445.

Chen, Z., Wu, Z., Lin, Z., Wang, S., Plant, C., Guo, W., 2023. AGNN: Alternating graph-regularized neural networks to alleviate over-smoothing. IEEE Trans. Neural Netw. Learn. Syst..

Chien, E., Peng, J., Li, P., Milenkovic, O., 2021. Adaptive universal generalized PageRank graph neural network. In: 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021. OpenReview.net.

Churcher, A., Ullah, R., Ahmad, J., Ur Rehman, S., Masood, F., Gogate, M., Alqahtani, F., Nour, B., Buchanan, W.J., 2021. An experimental analysis of attack classification using machine learning in IoT networks. Sensors 21 (2), 446.

Devendiran, R., Turukmane, A.V., 2024. Dugat-LSTM: Deep learning based network intrusion detection system using chaotic optimization strategy. Expert Syst. Appl. 245, 123027.

Duan, G., Lv, H., Wang, H., Feng, G., Li, X., 2024. Practical cyber attack detection with continuous temporal graph in dynamic network system. IEEE Trans. Inf. Forensics Secur..

ElSayed, M.S., Le-Khac, N.-A., Albahar, M.A., Jurcut, A., 2021. A novel hybrid model for intrusion detection systems in SDNs based on CNN and a new regularization technique. J. Netw. Comput. Appl. 191, 103160.

Fey, M., Lenssen, J.E., 2019. Fast graph representation learning with PyTorch geometric. arXiv preprint arXiv:1903.02428.

Gonçalves, L., Zanchettin, C., 2024. Detecting abnormal logins by discovering anomalous links via graph transformers. Comput. Secur. 103944.

Greff, K., Srivastava, R.K., Koutník, J., Steunebrink, B.R., Schmidhuber, J., 2016. LSTM: A search space odyssey. IEEE Trans. Neural Netw. Learn. Syst. 28 (10), 2222–2232.

Grohe, M., 2020. Word2vec, node2vec, graph2vec, x2vec: Towards a theory of vector embeddings of structured data. In: Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems. pp. 1–16.
Gu, J., Lu, S., 2021. An effective intrusion detection approach using SVM with naïve Bayes feature embedding. Comput. Secur. 103, 102158.
Haghighat, M.H., Li, J., 2021. Intrusion detection system using voting-based neural network. Tsinghua Sci. Technol. 26 (4), 484–495.
Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. Adv. Neural Inf. Process. Syst. 30.
He, D., Gu, H., Zhu, S., Chan, S., Guizani, M., 2023. A comprehensive detection method for the lateral movement stage of apt attacks. IEEE Internet Things J..
Heidari, A., Jabraeil Jamali, M.A., 2023. Internet of Things intrusion detection systems: a comprehensive review and future directions. Cluster Comput. 26 (6), 3753–3780.
Huang, S., Song, Y., Zhou, J., Lin, Z., 2024. Tailoring self-attention for graph via rooted subtrees. Adv. Neural Inf. Process. Syst. 36.
Imrana, Y., Xiang, Y., Ali, L., Abdul-Rauf, Z., 2021. A bidirectional LSTM deep learning approach for intrusion detection. Expert Syst. Appl. 185, 115524.
Jin, D., Lu, Y., Qin, J., Cheng, Z., Mao, Z., 2020. SwiftIDS: Real-time intrusion detection system based on LightGBM and parallel intrusion detection mechanism. Comput. Secur. 97, 101984.
Kanna, P.R., Santhi, P., 2021. Unified deep learning approach for efficient intrusion detection system using integrated spatial–temporal features. Knowl.-Based Syst. 226, 107132.
Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F., 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In: International Conference on Machine Learning. PMLR, pp. 5156–5165.
Laghrissi, F., Douzi, S., Douzi, K., Hssina, B., 2021. Intrusion detection systems using long short-term memory (LSTM). J. Big Data 8 (1), 65.
Lawal, M.A., Shaikh, R.A., Hassan, S.R., 2020. An anomaly mitigation framework for iot using fog computing. Electronics 9 (10), 1565.
Li, G., Müller, M., Ghanem, B., Koltun, V., 2021. Training graph neural networks with 1000 layers. In: International Conference on Machine Learning. PMLR, pp. 6437–6449.
Lo, W.W., Layeghy, S., Sarhan, M., Gallagher, M., Portmann, M., 2022. E-graphsage: A graph neural network based intrusion detection system for iot. In: NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–9.
Manocchio, L.D., Layeghy, S., Lo, W.W., Kulatilleke, G.K., Sarhan, M., Portmann, M., 2024. Flowtransformer: A transformer framework for flow-based network intrusion detection systems. Expert Syst. Appl. 241, 122564.
Nguyen, H., Kashef, R., 2023. TS-IDS: Traffic-aware self-supervised learning for IoT network intrusion detection. Knowl.-Based Syst. 279, 110966.
Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al., 2019. Pytorch: An imperative style, high-performance deep learning library. Adv. Neural Inf. Process. Syst. 32.
Peng, K., Li, M., Huang, H., Wang, C., Wan, S., Choo, K.-K.R., 2021. Security challenges and opportunities for smart contracts in Internet of Things: A survey. IEEE Internet Things J. 8 (15), 12004–12020.
Qureshi, S., et al., 2023. Limits of depth: Over-smoothing and over-squashing in GNNs. Big Data Min. Anal. 7 (1), 205–216.
Rani, B.S., Vairamuthu, S., Subramanian, S., 2024. Archimedes fire Hawk optimization enabled feature selection with deep maxout for network intrusion detection. Comput. Secur. 103751.
Rehman, M.U., Ahmadi, H., Hassan, W.U., 2024. FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE Computer Society, p. 139.
Sajjad, H., Durrani, N., Dalvi, F., 2022. Neuron-level interpretation of deep nlp models: A survey. Trans. Assoc. Comput. Linguist. 10, 1285–1303.
Sarhan, M., Layeghy, S., Moustafa, N., Portmann, M., 2021. Netflow datasets for machine learning-based network intrusion detection systems. In: Big Data Technologies and Applications: 10th EAI International Conference, BDTA 2020, and 13th EAI International Conference on Wireless Internet, WiCON 2020, Virtual Event, December 11, 2020, Proceedings 10. Springer, pp. 117–135.
Sarhan, M., Layeghy, S., Portmann, M., 2022. Towards a standard feature set for network intrusion detection system datasets. Mobile Netw. Appl. 1–14.
Shao, Y., Li, H., Gu, X., Yin, H., Li, Y., Miao, X., Zhang, W., Cui, B., Chen, L., 2024. Distributed graph neural network training: A survey. ACM Comput. Surv. 56 (8), 1–39.
Shi, H., Xu, M., Li, R., 2017. Deep learning for household load forecasting—A novel pooling deep RNN. IEEE Trans. Smart Grid 9 (5), 5271–5280.
Sohi, S.M., Seifert, J.-P., Ganji, F., 2021. RNNIDS: Enhancing network intrusion detection systems through deep learning. Comput. Secur. 102, 102151.
Sun, P., Liu, P., Li, Q., Liu, C., Lu, X., Hao, R., Chen, J., 2020. DL-IDS: Extracting features using CNN-LSTM hybrid network for intrusion detection system. Secur. Commun. Netw. 2020, 1–11.
Sun, K., Zhu, Z., Lin, Z., 2019. Adagcn: Adaboosting graph convolutional networks into deep models. arXiv preprint arXiv:1908.05081.
Talukder, M.A., Islam, M.M., Uddin, M.A., Hasan, K.F., Sharmin, S., Alyami, S.A., Moni, M.A., 2024. Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction. J. Big Data 11 (1), 33.
Thakkar, A., Lohiya, R., 2022. A survey on intrusion detection system: feature selection, model, performance measures, application perspective, challenges, and future research directions. Artif. Intell. Rev. 55 (1), 453–563.
Torres, L.H., Ribeiro, B., Arrais, J.P., 2023. Few-shot learning with transformers via graph embeddings for molecular property prediction. Expert Syst. Appl. 225, 120005.
Treviso, M., Lee, J.-U., Ji, T., Aken, B.v., Cao, Q., Ciosici, M.R., Hassid, M., Heafield, K., Hooker, S., Raffel, C., et al., 2023. Efficient methods for natural language processing: A survey. Trans. Assoc. Comput. Linguist. 11, 826–860.
Turukmane, A.V., Devendiran, R., 2024. M-MultiSVM: An efficient feature selection assisted network intrusion detection system using machine learning. Comput. Secur. 137, 103587.
Van Rossum, G., Drake, Jr., F.L., 1995. Python tutorial.
Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., et al., 2017. Graph attention networks. Statistics 1050 (20), 10–48550.
Wang, M.Y., 2019. Deep graph library: Towards efficient and scalable deep learning on graphs. In: ICLR Workshop on Representation Learning on Graphs and Manifolds.
Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., Weinberger, K., 2019. Simplifying graph convolutional networks. In: International Conference on Machine Learning. PMLR, pp. 6861–6871.
Xhonneux, L.-P., Qu, M., Tang, J., 2020. Continuous graph neural networks. In: International Conference on Machine Learning. PMLR, pp. 10432–10441.
Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K.-i., Jegelka, S., 2018. Representation learning on graphs with jumping knowledge networks. In: International Conference on Machine Learning. PMLR, pp. 5453–5462.
Xu, R., Wu, G., Wang, W., Gao, X., He, A., Zhang, Z., 2024. Applying self-supervised learning to network intrusion detection for network flows with graph neural network. Comput. Netw. 248, 110495.
Xu, M., Yoon, S., Fuentes, A., Park, D.S., 2023. A comprehensive survey of image augmentation techniques for deep learning. Pattern Recognit. 137, 109347.
Yi, T., Chen, X., Zhu, Y., Ge, W., Han, Z., 2023. Review on the application of deep learning in network attack detection. J. Netw. Comput. Appl. 212, 103580.
Zhang, S., Li, X., Zong, M., Zhu, X., Wang, R., 2017. Efficient kNN classification with different numbers of nearest neighbors. IEEE Trans. Neural Netw. Learn. Syst. 29 (5), 1774–1785.
Zhang, J., Ling, Y., Fu, X., Yang, X., Xiong, G., Zhang, R., 2020. Model of the intrusion detection system based on the integration of spatial-temporal features. Comput. Secur. 89, 101681.
Zhao, J., Dong, Y., Ding, M., Kharlamov, E., Tang, J., 2021. Adaptive diffusion in graph neural networks. Adv. Neural Inf. Process. Syst. 34, 23321–23333.
Zhong, L., Wu, J., Li, Q., Peng, H., Wu, X., 2023. A comprehensive survey on automatic knowledge graph construction. ACM Comput. Surv. 56 (4), 1–62.

Ping Deng received his bachelor's degree. He is currently a postgraduate in Electronic Information at the College of Computer Science, Sichuan Normal University, China. His research interests include network security and deep learning.

Yong Huang, Ph.D., Senior Engineer, graduated from the University of Electronic Science and Technology of China. His research interests include network security, information security, and open-source intelligence analysis. He has been dedicated to the research and practice of information security technology and product industrialization for many years. He holds 17 national invention patents and has led and guided teams to develop over 30 network security products. He has received three provincial and ministerial science and technology progress awards and has led or participated in over 10 provincial and ministerial information security industrialization projects.